Mane-Kelsa - Source Code

React + TypeScript + Vite + Tailwind + Lovable Cloud (Supabase)

Files included:

- index.html
- package.json
- vite.config.ts
- tailwind.config.ts
- src/main.tsx
- src/App.tsx
- src/App.css
- src/index.css
- src/pages/Index.tsx
- src/pages/Auth.tsx
- src/pages/Dashboard.tsx
- src/pages/NotFound.tsx
- src/lib/i18n.ts
- src/lib/utils.ts
- src/components/NavLink.tsx
- src/hooks/use-mobile.tsx
- src/hooks/use-toast.ts
- src/integrations/supabase/client.ts
- supabase/migrations/20260504115455_970c4592-b84e-4b74-a0db-ea50c842b695.sql
- supabase/migrations/20260504115519_2f96c0fb-1030-4969-82ef-f85743725ba2.sql
- supabase/migrations/20260504122914_def23943-92ee-4874-879d-aff1070e0892.sql
- supabase/migrations/20260504122939_d771950b-e4f2-4df2-ae01-ebd60cb30140.sql
- supabase/migrations/20260507001640_f7afb6b7-dd33-4965-aa15-e762ae504811.sql
- supabase/migrations/20260507001708_e49da20b-9188-496c-87ee-d473d5276472.sql
- supabase/migrations/20260515172646_07adf3bd-0835-4db0-89fe-682d48534cdc.sql
- supabase/migrations/20260515172705_5d4f314d-3eef-401e-82fe-2087e16dcd20.sql

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- TODO: Set the document title to the name of your application -->
    <title>shilpa</title>
    <meta name="description" content="Creative Animation Studio empowers users to create animated projects wit
h a focus on creativity and localization.">
    <meta name="author" content="Lovable" />

    <!-- TODO: Update og:title to match your application name -->


    <meta property="og:type" content="website" />
    <meta property="og:image" content="https://pub-bb2e103a32db4e198524a2e9ed8f35b4.r2.dev/04f8075f-8446-4a31-
b588-94e50411ad15/id-preview-d9b73715--b92f5ba4-ab8d-462c-bcb4-c4766483a480.lovable.app-1777897890001.png">

    <meta name="twitter:card" content="summary_large_image" />
    <meta name="twitter:site" content="@Lovable" />
    <meta name="twitter:image" content="https://pub-bb2e103a32db4e198524a2e9ed8f35b4.r2.dev/04f8075f-8446-4a31
-b588-94e50411ad15/id-preview-d9b73715--b92f5ba4-ab8d-462c-bcb4-c4766483a480.lovable.app-1777897890001.png">
    <meta property="og:title" content="shilpa">
  <meta name="twitter:title" content="shilpa">
  <meta property="og:description" content="Creative Animation Studio empowers users to create animated project
s with a focus on creativity and localization.">
  <meta name="twitter:description" content="Creative Animation Studio empowers users to create animated projec
ts with a focus on creativity and localization.">
  <link rel="icon" type="image/x-icon" href="/favicon.ico">
</head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

package.json

```
{
  "name": "vite_react_shadcn_ts",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "build:dev": "vite build --mode development",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest run",
    "test:watch": "vitest"
  },
  "dependencies": {
    "@hookform/resolvers": "^3.10.0",
    "@radix-ui/react-accordion": "^1.2.11",
    "@radix-ui/react-alert-dialog": "^1.1.14",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.2",
    "@radix-ui/react-collapsible": "^1.1.11",
    "@radix-ui/react-context-menu": "^2.2.15",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-dropdown-menu": "^2.1.15",
    "@radix-ui/react-hover-card": "^1.1.14",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.13",
    "@radix-ui/react-popover": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.7",
    "@radix-ui/react-scroll-area": "^1.2.9",
    "@radix-ui/react-select": "^2.2.5",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.5",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.5",
    "@radix-ui/react-tabs": "^1.1.12",
    "@radix-ui/react-toast": "^1.2.14",
    "@radix-ui/react-toggle": "^1.1.9",
    "@radix-ui/react-toggle-group": "^1.1.10",
    "@radix-ui/react-tooltip": "^1.2.7",
    "@supabase/supabase-js": "^2.105.1",
    "@tanstack/react-query": "^5.83.0",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^3.6.0",
    "embla-carousel-react": "^8.6.0",
    "input-otp": "^1.4.2",
    "lucide-react": "^0.462.0",
    "next-themes": "^0.3.0",
    "react": "^18.3.1",
    "react-day-picker": "^8.10.1",
    "react-dom": "^18.3.1",
    "react-hook-form": "^7.61.1",
    "react-resizable-panels": "^2.1.9",
    "react-router-dom": "^6.30.1",
    "recharts": "^2.15.4",
    "sonner": "^1.7.4",
    "tailwind-merge": "^2.6.0",
    "tailwindcss-animate": "^1.0.7",
    "vaul": "^0.9.9",
    "zod": "^3.25.76"
  },
  "devDependencies": {
    "@eslint/js": "^9.32.0",
    "@tailwindcss/typography": "^0.5.16",
    "@testing-library/jest-dom": "^6.6.0",
    "@testing-library/react": "^16.0.0",
    "@types/node": "^22.16.5",
    "@types/react": "^18.3.23",
    "@types/react-dom": "^18.3.7",
    "@vitejs/plugin-react-swc": "^3.11.0",
    "autoprefixer": "^10.4.21",
```

```
    "eslint": "^9.32.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^15.15.0",
    "jsdom": "^20.0.3",
    "lovable-tagger": "^1.1.13",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.17",
    "typescript": "^5.8.3",
    "typescript-eslint": "^8.38.0",
    "vite": "^5.4.19",
    "vitest": "^3.2.4"
  }
}
```

vite.config.ts

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react-swc";
import path from "path";
import { componentTagger } from "lovable-tagger";

// https://vitejs.dev/config/
export default defineConfig(({ mode }) => ({
  server: {
    host: "::",
    port: 8080,
    hmr: {
      overlay: false,
    },
  },
  plugins: [react(), mode === "development" && componentTagger()].filter(Boolean),
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
    dedupe: ["react", "react-dom", "react/jsx-runtime", "react/jsx-dev-runtime", "@tanstack/react-query", "@ta
nstack/query-core"],
  },
}));
```

tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: ["./pages/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "./app/**/*.{ts,tsx}", "./src/**/*.{ts,tsx}
"],
  prefix: "",
  theme: {
    container: {
      center: true,
      padding: "2rem",
      screens: {
        "2xl": "1400px",
      },
    },
    extend: {
      colors: {
        border: "hsl(var(--border))",
        input: "hsl(var(--input))",
        ring: "hsl(var(--ring))",
        background: "hsl(var(--background))",
        foreground: "hsl(var(--foreground))",
        primary: {
          DEFAULT: "hsl(var(--primary))",
          foreground: "hsl(var(--primary-foreground))",
        },
        secondary: {
          DEFAULT: "hsl(var(--secondary))",
          foreground: "hsl(var(--secondary-foreground))",
        },
        destructive: {
          DEFAULT: "hsl(var(--destructive))",
          foreground: "hsl(var(--destructive-foreground))",
        },
        muted: {
          DEFAULT: "hsl(var(--muted))",
          foreground: "hsl(var(--muted-foreground))",
        },
        accent: {
          DEFAULT: "hsl(var(--accent))",
          foreground: "hsl(var(--accent-foreground))",
        },
        popover: {
          DEFAULT: "hsl(var(--popover))",
          foreground: "hsl(var(--popover-foreground))",
        },
        card: {
          DEFAULT: "hsl(var(--card))",
          foreground: "hsl(var(--card-foreground))",
        },
        sidebar: {
          DEFAULT: "hsl(var(--sidebar-background))",
          foreground: "hsl(var(--sidebar-foreground))",
          primary: "hsl(var(--sidebar-primary))",
          "primary-foreground": "hsl(var(--sidebar-primary-foreground))",
          accent: "hsl(var(--sidebar-accent))",
          "accent-foreground": "hsl(var(--sidebar-accent-foreground))",
          border: "hsl(var(--sidebar-border))",
          ring: "hsl(var(--sidebar-ring))",
        },
      },
      borderRadius: {
        lg: "var(--radius)",
        md: "calc(var(--radius) - 2px)",
        sm: "calc(var(--radius) - 4px)",
      },
      keyframes: {
        "accordion-down": {
          from: {
            height: "0",
          },
          to: {
            height: "var(--radix-accordion-content-height)",
          },
        },
        "accordion-up": {
```

```
          from: {
            height: "var(--radix-accordion-content-height)",
          },
          to: {
            height: "0",
          },
        },
      },
      animation: {
        "accordion-down": "accordion-down 0.2s ease-out",
        "accordion-up": "accordion-up 0.2s ease-out",
      },
    },
  },
  plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

src/main.tsx

```
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import "./index.css";

createRoot(document.getElementById("root")!).render(<App />);
```

src/App.tsx

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Route, Routes } from "react-router-dom";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { Toaster } from "@/components/ui/toaster";
import { TooltipProvider } from "@/components/ui/tooltip";
import Index from "./pages/Index.tsx";
import NotFound from "./pages/NotFound.tsx";
import Auth from "./pages/Auth.tsx";
import Dashboard from "./pages/Dashboard.tsx";

const queryClient = new QueryClient();

const App = () => (
  <QueryClientProvider client={queryClient}>
    <TooltipProvider>
      <Toaster />
      <Sonner />
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Index />} />
          <Route path="/auth" element={<Auth />} />
          <Route path="/dashboard" element={<Dashboard />} />
          {/* ADD ALL CUSTOM ROUTES ABOVE THE CATCH-ALL "*" ROUTE */}
          <Route path="*" element={<NotFound />} />
        </Routes>
      </BrowserRouter>
    </TooltipProvider>
  </QueryClientProvider>
);

export default App;
```

src/App.css

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 35 40% 97%;
    --foreground: 25 35% 12%;

    --card: 0 0% 100%;
    --card-foreground: 25 35% 12%;

    --popover: 0 0% 100%;
    --popover-foreground: 25 35% 12%;

    --primary: 18 88% 52%;
    --primary-foreground: 35 40% 98%;
    --primary-glow: 32 95% 60%;

    --secondary: 145 55% 38%;
    --secondary-foreground: 35 40% 98%;

    --muted: 35 30% 92%;
    --muted-foreground: 25 15% 40%;

    --accent: 45 95% 55%;
    --accent-foreground: 25 35% 12%;

    --destructive: 0 84% 60%;
    --destructive-foreground: 35 40% 98%;

    --border: 30 20% 86%;
    --input: 30 20% 86%;
    --ring: 18 88% 52%;

    --radius: 1rem;

    --gradient-hero: linear-gradient(135deg, hsl(18 88% 52%) 0%, hsl(32 95% 60%) 50%, hsl(45 95% 55%) 100%);
    --gradient-warm: linear-gradient(180deg, hsl(35 40% 97%) 0%, hsl(35 50% 93%) 100%);
    --gradient-card: linear-gradient(145deg, hsl(0 0% 100%) 0%, hsl(35 40% 97%) 100%);
    --gradient-green: linear-gradient(135deg, hsl(145 55% 38%) 0%, hsl(160 60% 45%) 100%);

    --shadow-warm: 0 20px 50px -15px hsl(18 88% 52% / 0.35);
    --shadow-soft: 0 10px 30px -10px hsl(25 35% 12% / 0.15);
    --shadow-glow: 0 0 60px hsl(32 95% 60% / 0.5);

    --transition-smooth: cubic-bezier(0.4, 0, 0.2, 1);
    --transition-bounce: cubic-bezier(0.34, 1.56, 0.64, 1);

    --sidebar-background: 0 0% 98%;
    --sidebar-foreground: 240 5.3% 26.1%;
    --sidebar-primary: 240 5.9% 10%;
    --sidebar-primary-foreground: 0 0% 98%;
    --sidebar-accent: 240 4.8% 95.9%;
    --sidebar-accent-foreground: 240 5.9% 10%;
    --sidebar-border: 220 13% 91%;
    --sidebar-ring: 217.2 91.2% 59.8%;
  }
}

@layer base {
  * {
    @apply border-border;
  }
  body {
    @apply bg-background text-foreground;
    font-family: 'Inter', system-ui, sans-serif;
  }
}

@layer utilities {
  .bg-gradient-hero { background: var(--gradient-hero); }
  .bg-gradient-warm { background: var(--gradient-warm); }
  .bg-gradient-card { background: var(--gradient-card); }
  .bg-gradient-green { background: var(--gradient-green); }
  .shadow-warm { box-shadow: var(--shadow-warm); }
```

```
  .shadow-soft { box-shadow: var(--shadow-soft); }
  .shadow-glow { box-shadow: var(--shadow-glow); }
  .text-gradient {
    background: var(--gradient-hero);
    -webkit-background-clip: text;
    background-clip: text;
    color: transparent;
  }
}

@keyframes float {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-12px); }
}
@keyframes pulse-ring {
  0% { transform: scale(0.8); opacity: 0.8; }
  100% { transform: scale(2.4); opacity: 0; }
}
@keyframes slide-up {
  from { opacity: 0; transform: translateY(40px); }
  to { opacity: 1; transform: translateY(0); }
}
@keyframes fade-in {
  from { opacity: 0; }
  to { opacity: 1; }
}
@keyframes shimmer {
  0% { background-position: -200% 0; }
  100% { background-position: 200% 0; }
}
@keyframes wiggle {
  0%, 100% { transform: rotate(0); }
  25% { transform: rotate(-3deg); }
  75% { transform: rotate(3deg); }
}
@keyframes blob {
  0%, 100% { border-radius: 60% 40% 30% 70% / 60% 30% 70% 40%; }
  50% { border-radius: 30% 60% 70% 40% / 50% 60% 30% 60%; }
}

.animate-float { animation: float 4s ease-in-out infinite; }
.animate-pulse-ring { animation: pulse-ring 2s cubic-bezier(0.4, 0, 0.2, 1) infinite; }
.animate-slide-up { animation: slide-up 0.8s var(--transition-smooth) both; }
.animate-fade-in { animation: fade-in 1s ease-out both; }
.animate-wiggle { animation: wiggle 1.5s ease-in-out infinite; }
.animate-blob { animation: blob 8s ease-in-out infinite; }
```

src/pages/Index.tsx

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { Card } from "@/components/ui/card";
import {
  MapPin,
  Phone,
  ThumbsUp,
  ToggleRight,
  Sparkles,
  Wifi,
  TrendingUp,
  Shield,
  Building2,
  Users,
  Star,
  ArrowRight,
  Languages,
} from "lucide-react";
import heroWorkers from "@/assets/hero-workers.png";

type Lang = "kn" | "en";

const t = {
  kn: {
    brand: "[redacted]",
    nav: { features: "[redacted]", how: "[redacted]", impact: "[redacted]", start: "[redacted]" },
    hero: {
      live: "[redacted]",
      titleA: "[redacted]", titleB: "[redacted]", titleC: " [redacted].",
      desc: '[redacted]. [redacted] "[redacted]" [redacted]. [redacted].',
      cta1: "[redacted]",
      cta2: "[redacted]",
      rating: "[redacted]",
      ratingDesc: "[redacted]",
      status: "[redacted]", available: "[redacted]",
      ratingLabel: "[redacted]", trust: "[redacted]",
      alt: "[redacted]",
    },
    feat: {
      tag: "[redacted]",
      hA: "", hB: "[redacted]", hC: "[redacted]",
      desc: "[redacted], [redacted], [redacted] UI.",
      items: [
        { title: "[redacted]", desc: "[redacted]." },
        { title: "[redacted]", desc: '"[redacted]" — [redacted].' },
        { title: "[redacted]", desc: "[redacted]." },
        { title: "[redacted]", desc: "[redacted]." },
      ],
    },
    how: {
      h: "[redacted]",
      desc: "[redacted]. [redacted].",
      nums: ["[redacted]", "[redacted]", "[redacted]"],
      items: [
        { title: "[redacted]", desc: "[redacted]/[redacted]." },
        { title: "[redacted]", desc: "[redacted] ACTION_DIAL [redacted]." },
        { title: "[redacted]", desc: "[redacted] — [redacted]." }
,
      ],
    },
    impact: {
      tag: "[redacted]",
      hA: "", hB: "[redacted]", hC: "[redacted]",
      desc: "[redacted].
,
      items: [
        { title: "[redacted]", desc: "[redacted]." },
        { title: "[redacted]", desc: "[redacted]." },
        { title: "[redacted]", desc: "[redacted]." },
      ],
    },
    cta: {
      h: "[redacted]",
```

```
      desc: "[redacted] [redacted] — [redacted] [redacted] [redacted] [redacted], [redacted] [redacted] [redacted], [redacted] [redacted] [redacted]
[redacted].",
      b1: "[redacted] [redacted] [redacted]", b2: "[redacted]",
    },
    footer: { copy: "© [redacted] [redacted]-[redacted] · [redacted] VTU [redacted]", made: ["[redacted]", "[redacted] [redacted]
[redacted]"] },
    toggle: "English",
  },
  en: {
    brand: "Mane-Kelsa",
    nav: { features: "Features", how: "How it works", impact: "Impact", start: "Get Started" },
    hero: {
      live: "LIVE · Local Work Directory",
      titleA: "Your ", titleB: "Digital Naka", titleC: " for everyday work.",
      desc: 'Mane-Kelsa instantly connects domestic workers and gardeners with nearby families. No more "word
of mouth". One tap to find work today.',
      cta1: "I'm Available Today",
      cta2: "Find Workers",
      rating: "4.9",
      ratingDesc: "2,400+ workers across Karnataka towns",
      status: "Status", available: "Available Now",
      ratingLabel: "Rating", trust: "98% Trusted",
      alt: "Local worker ready for the day's work",
    },
    feat: {
      tag: "APP USAGE",
      hA: "Built for ", hB: "Real Bharat", hC: "",
      desc: "Big buttons, clear text, and bilingual UI for semi-literate users.",
      items: [
        { title: "Worker Profile", desc: "Skills, photo and daily rate." },
        { title: "Availability Switch", desc: '"I\'m available today" — one switch.' },
        { title: "Direct Call", desc: "Call workers in a single tap." },
        { title: "Trust Rating", desc: "Simple thumbs up from owners." },
      ],
    },
    how: {
      h: "Technology that just works",
      desc: "Real-time sync. Offline ready. One-tap calling.",
      nums: ["01", "02", "03"],
      items: [
        { title: "Real-time Sync", desc: "Firebase keeps online/offline status fresh for everyone instantly."
},
        { title: "Direct Intent Call", desc: "Call via ACTION_DIAL without leaving the app." },
        { title: "Nearest Street", desc: "Sorted by your area — find help right on your street." },
      ],
    },
    impact: {
      tag: "IMPACT GOALS",
      hA: "Digitizing the ", hB: "Unorganized Sector", hC: "",
      desc: "Mane-Kelsa is more than an app — it's economic security for thousands of workers in tier-3 towns.
",
      items: [
        { title: "Unorganized Sector Growth", desc: "Digitizing local labor for better, fair wages." },
        { title: "Economic Security", desc: "A steady flow of nearby opportunities." },
        { title: "Urban-Rural Bridge", desc: "Professionalising services in growing towns." },
      ],
    },
    cta: {
      h: "Join your local Naka, digitally.",
      desc: "From cleaning to gardening — list your skills, set your day rate, and start earning today.",
      b1: "Download the App", b2: "For Owners",
    },
    footer: { copy: "© 2026 Mane-Kelsa · MindMatrix VTU Internship", made: ["Made with", "in Karnataka"] },
    toggle: "[redacted]",
  },
} as const;

const Index = () => {
  const [lang, setLang] = useState<Lang>("kn");
  const c = t[lang];
  const fontClass = lang === "kn" ? "font-kannada" : "";

  return (
    <div className={`min-h-screen bg-gradient-warm overflow-hidden ${fontClass}`}>
      {/* Nav */}
      <nav className="container flex items-center justify-between py-6 relative z-10">
        <div className="flex items-center gap-2 animate-fade-in">
          <div className="w-10 h-10 rounded-xl bg-gradient-hero flex items-center justify-center shadow-warm">
```

```
            <Sparkles className="w-5 h-5 text-primary-foreground" />
          </div>
          <span className="text-xl font-bold">{c.brand}</span>
        </div>
        <div className="hidden md:flex items-center gap-8 text-sm font-medium">
          <a href="#features" className="hover:text-primary transition-colors">{c.nav.features}</a>
          <a href="#how" className="hover:text-primary transition-colors">{c.nav.how}</a>
          <a href="#impact" className="hover:text-primary transition-colors">{c.nav.impact}</a>
        </div>
        <div className="flex items-center gap-2">
          <Button
            variant="outline"
            size="sm"
            onClick={() => setLang(lang === "kn" ? "en" : "kn")}
            aria-label="Toggle language"
          >
            <Languages className="w-4 h-4" />
            {c.toggle}
          </Button>
          <Button variant="hero" size="sm" asChild><Link to="/auth">{c.nav.start}</Link></Button>
        </div>
      </nav>

      {/* Hero */}
      <section className="container relative pt-12 pb-24">
        <div className="absolute top-20 -left-20 w-72 h-72 bg-primary/20 animate-blob blur-3xl" />
        <div className="absolute top-40 right-0 w-96 h-96 bg-secondary/20 animate-blob blur-3xl" style={{ anim
ationDelay: "2s" }} />

        <div className="relative grid lg:grid-cols-2 gap-12 items-center">
          <div className="space-y-8 animate-slide-up">
            <div className="inline-flex items-center gap-2 px-4 py-2 rounded-full bg-card border shadow-soft">
              <span className="relative flex h-2 w-2">
                <span className="animate-pulse-ring absolute inline-flex h-full w-full rounded-full bg-seconda
ry" />
                <span className="relative inline-flex rounded-full h-2 w-2 bg-secondary" />
              </span>
              <span className="text-xs font-semibold">{c.hero.live}</span>
            </div>

            <h1 className="text-5xl md:text-7xl font-extrabold leading-[1.15] tracking-tight">
              {c.hero.titleA}<span className="text-gradient">{c.hero.titleB}</span>{c.hero.titleC}
            </h1>

            <p className="text-lg text-muted-foreground max-w-lg">{c.hero.desc}</p>

            <div className="flex flex-wrap gap-4">
              <Button variant="hero" size="xl" className="group" asChild>
                <Link to="/auth">
                  {c.hero.cta1}
                  <ArrowRight className="group-hover:translate-x-1 transition-transform" />
                </Link>
              </Button>
              <Button variant="outline" size="xl" asChild>
                <Link to="/auth">{c.hero.cta2}</Link>
              </Button>
            </div>

            <div className="flex items-center gap-6 pt-4">
              <div className="flex -space-x-3">
                {[1, 2, 3, 4].map((i) => (
                  <div key={i} className="w-10 h-10 rounded-full bg-gradient-hero border-2 border-background s
hadow-soft" />
                ))}
              </div>
              <div className="text-sm">
                <div className="font-bold flex items-center gap-1">
                  {c.hero.rating} <Star className="w-4 h-4 fill-accent text-accent" />
                </div>
                <div className="text-muted-foreground">{c.hero.ratingDesc}</div>
              </div>
            </div>
          </div>

          <div className="relative animate-fade-in">
            <div className="absolute inset-0 bg-gradient-hero rounded-[3rem] blur-3xl opacity-30 animate-float
" />
            <div className="relative bg-gradient-card rounded-[3rem] p-8 shadow-warm">
```

```
              <img
                src={heroWorkers}
                alt={c.hero.alt}
                width={1024}
                height={1024}
                className="w-full h-auto animate-float"
              />
              <Card className="absolute -left-4 top-12 p-4 shadow-warm bg-card animate-float" style={{ animati
onDelay: "1s" }}>
                <div className="flex items-center gap-3">
                  <div className="w-10 h-10 rounded-full bg-secondary/20 flex items-center justify-center">
                    <ToggleRight className="w-5 h-5 text-secondary" />
                  </div>
                  <div>
                    <div className="text-xs text-muted-foreground">{c.hero.status}</div>
                    <div className="text-sm font-bold text-secondary">{c.hero.available}</div>
                  </div>
                </div>
              </Card>
              <Card className="absolute -right-4 bottom-16 p-4 shadow-warm bg-card animate-float" style={{ ani
mationDelay: "2s" }}>
                <div className="flex items-center gap-3">
                  <div className="w-10 h-10 rounded-full bg-accent/30 flex items-center justify-center">
                    <ThumbsUp className="w-5 h-5 text-primary" />
                  </div>
                  <div>
                    <div className="text-xs text-muted-foreground">{c.hero.ratingLabel}</div>
                    <div className="text-sm font-bold">{c.hero.trust}</div>
                  </div>
                </div>
              </Card>
            </div>
          </div>
        </div>
      </section>

      {/* Features */}
      <section id="features" className="container py-24">
        <div className="text-center max-w-2xl mx-auto mb-16 animate-slide-up">
          <div className="inline-block px-3 py-1 rounded-full bg-primary/10 text-primary text-xs font-bold mb-
4">
            {c.feat.tag}
          </div>
          <h2 className="text-4xl md:text-5xl font-extrabold mb-4">
            {c.feat.hA}<span className="text-gradient">{c.feat.hB}</span>{c.feat.hC}
          </h2>
          <p className="text-muted-foreground">{c.feat.desc}</p>
        </div>

        <div className="grid md:grid-cols-2 lg:grid-cols-4 gap-6">
          {c.feat.items.map((f, i) => {
            const icons = [Users, ToggleRight, Phone, ThumbsUp];
            const colors = ["from-primary to-primary-glow", "from-secondary to-secondary", "from-accent to-pri
mary", "from-primary-glow to-accent"];
            const Icon = icons[i];
            return (
              <Card
                key={i}
                className="group p-6 bg-gradient-card border-2 hover:border-primary/40 hover:-translate-y-2 tr
ansition-all duration-500 shadow-soft hover:shadow-warm cursor-pointer"
              >
                <div className={`w-14 h-14 rounded-2xl bg-gradient-to-br ${colors[i]} flex items-center justif
y-center mb-5 group-hover:animate-wiggle shadow-soft`}>
                  <Icon className="w-7 h-7 text-primary-foreground" />
                </div>
                <h3 className="text-lg font-bold mb-2">{f.title}</h3>
                <p className="text-sm text-muted-foreground">{f.desc}</p>
              </Card>
            );
          })}
        </div>
      </section>

      {/* How */}
      <section id="how" className="bg-gradient-hero py-24 relative overflow-hidden">
        <div className="absolute inset-0 opacity-10">
          <div className="absolute top-10 left-10 w-64 h-64 rounded-full bg-primary-foreground animate-blob" /
>
```

```
          <div className="absolute bottom-10 right-10 w-80 h-80 rounded-full bg-primary-foreground animate-blo
b" style={{ animationDelay: "3s" }} />
        </div>

        <div className="container relative">
          <div className="text-center mb-16 text-primary-foreground">
            <h2 className="text-4xl md:text-5xl font-extrabold mb-4">{c.how.h}</h2>
            <p className="opacity-90">{c.how.desc}</p>
          </div>

          <div className="grid md:grid-cols-3 gap-6">
            {c.how.items.map((s, i) => {
              const icons = [Wifi, Phone, MapPin];
              const Icon = icons[i];
              return (
                <Card key={i} className="p-8 bg-card/95 backdrop-blur border-0 shadow-warm hover:scale-105 tra
nsition-transform duration-500">
                  <div className="text-6xl font-extrabold text-gradient mb-4">{c.how.nums[i]}</div>
                  <Icon className="w-8 h-8 text-primary mb-3" />
                  <h3 className="text-xl font-bold mb-2">{s.title}</h3>
                  <p className="text-sm text-muted-foreground">{s.desc}</p>
                </Card>
              );
            })}
          </div>
        </div>
      </section>

      {/* Impact */}
      <section id="impact" className="container py-24">
        <div className="grid lg:grid-cols-2 gap-12 items-center">
          <div className="space-y-6 animate-slide-up">
            <div className="inline-block px-3 py-1 rounded-full bg-secondary/15 text-secondary text-xs font-bo
ld">
              {c.impact.tag}
            </div>
            <h2 className="text-4xl md:text-5xl font-extrabold">
              {c.impact.hA}<span className="text-gradient">{c.impact.hB}</span>{c.impact.hC}
            </h2>
            <p className="text-muted-foreground text-lg">{c.impact.desc}</p>
          </div>

          <div className="grid gap-4">
            {c.impact.items.map((g, i) => {
              const icons = [TrendingUp, Shield, Building2];
              const colors = ["bg-primary", "bg-secondary", "bg-accent"];
              const Icon = icons[i];
              return (
                <Card key={i} className="p-6 flex items-start gap-4 hover:translate-x-2 transition-transform d
uration-300 shadow-soft hover:shadow-warm">
                  <div className={`w-12 h-12 rounded-xl ${colors[i]} flex items-center justify-center shrink-0
 shadow-soft`}>
                    <Icon className="w-6 h-6 text-primary-foreground" />
                  </div>
                  <div>
                    <h3 className="font-bold mb-1">{g.title}</h3>
                    <p className="text-sm text-muted-foreground">{g.desc}</p>
                  </div>
                </Card>
              );
            })}
          </div>
        </div>
      </section>

      {/* CTA */}
      <section className="container pb-24">
        <Card className="relative overflow-hidden p-12 md:p-16 bg-gradient-hero border-0 shadow-warm text-cent
er">
          <div className="absolute -top-20 -right-20 w-64 h-64 bg-primary-foreground/10 animate-blob" />
          <div className="absolute -bottom-20 -left-20 w-64 h-64 bg-primary-foreground/10 animate-blob" style=
{{ animationDelay: "2s" }} />
          <div className="relative space-y-6 text-primary-foreground">
            <h2 className="text-4xl md:text-6xl font-extrabold">{c.cta.h}</h2>
            <p className="opacity-90 max-w-2xl mx-auto text-lg">{c.cta.desc}</p>
            <div className="flex flex-wrap gap-4 justify-center pt-4">
              <Button variant="glow" size="xl" className="bg-card text-foreground hover:bg-card/90" asChild>
                <Link to="/auth">{c.cta.b1}</Link>
```

```
              </Button>
              <Button variant="outline" size="xl" className="bg-transparent border-primary-foreground/40 text-
primary-foreground hover:bg-primary-foreground/10" asChild>
                <Link to="/auth">{c.cta.b2}</Link>
              </Button>
            </div>
          </div>
        </Card>
      </section>

      <footer className="container py-8 border-t text-sm text-muted-foreground flex flex-wrap items-center jus
tify-between gap-4">
        <div>{c.footer.copy}</div>
        <div className="flex items-center gap-2">
          {c.footer.made[0]} <span className="text-primary animate-wiggle inline-block">♥</span> {c.footer.mad
e[1]}
        </div>
      </footer>
    </div>
  );
};

export default Index;
```

src/pages/Auth.tsx

```
import { useEffect, useState } from "react";
import { useNavigate, Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card } from "@/components/ui/card";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { toast } from "sonner";
import { Languages, Sparkles, ArrowLeft } from "lucide-react";
import { tAuth, type Lang } from "@/lib/i18n";

const Auth = () => {
  const nav = useNavigate();
  const [lang, setLang] = useState<Lang>("kn");
  const c = tAuth[lang];
  const [loading, setLoading] = useState(false);

  // sign in
  const [siEmail, setSiEmail] = useState("");
  const [siPass, setSiPass] = useState("");

  // sign up
  const [suEmail, setSuEmail] = useState("");
  const [suPass, setSuPass] = useState("");
  const [name, setName] = useState("");
  const [phone, setPhone] = useState("");
  const [area, setArea] = useState("");
  const [role, setRole] = useState<"worker" | "owner">("worker");

  useEffect(() => {
    supabase.auth.getSession().then(({ data }) => {
      if (data.session) nav("/dashboard", { replace: true });
    });
    const { data: sub } = supabase.auth.onAuthStateChange((_e, session) => {
      if (session) nav("/dashboard", { replace: true });
    });
    return () => sub.subscription.unsubscribe();
  }, [nav]);

  const handleSignIn = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    const { error } = await supabase.auth.signInWithPassword({ email: siEmail, password: siPass });
    setLoading(false);
    if (error) toast.error(error.message);
  };

  const handleSignUp = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    const { error } = await supabase.auth.signUp({
      email: suEmail,
      password: suPass,
      options: {
        emailRedirectTo: `${window.location.origin}/dashboard`,
        data: { full_name: name, phone, area, role },
      },
    });
    setLoading(false);
    if (error) toast.error(error.message);
    else toast.success(lang === "kn" ? "■■■■ ■■■■■■■■■■!" : "Account created!");
  };

  const fontClass = lang === "kn" ? "font-kannada" : "";

  return (
    <div className={`min-h-screen bg-gradient-warm flex flex-col ${fontClass}`}>
      <nav className="container flex items-center justify-between py-6">
        <Link to="/" className="flex items-center gap-2">
          <div className="w-10 h-10 rounded-xl bg-gradient-hero flex items-center justify-center shadow-warm">
            <Sparkles className="w-5 h-5 text-primary-foreground" />
          </div>
          <span className="text-xl font-bold">■■■-■■■■</span>
        </Link>
        <div className="flex gap-2">
```

```
          <Button variant="outline" size="sm" asChild>
            <Link to="/"><ArrowLeft className="w-4 h-4" />{c.back}</Link>
          </Button>
          <Button variant="outline" size="sm" onClick={() => setLang(lang === "kn" ? "en" : "kn")}>
            <Languages className="w-4 h-4" />{lang === "kn" ? "English" : "■■■■■"}
          </Button>
        </div>
      </nav>

      <main className="flex-1 container flex items-center justify-center pb-12">
        <Card className="w-full max-w-md p-8 shadow-warm bg-gradient-card border-2 animate-slide-up">
          <div className="text-center mb-6">
            <h1 className="text-3xl font-extrabold mb-1">{c.title}</h1>
            <p className="text-sm text-muted-foreground">{c.sub}</p>
          </div>

          <Tabs defaultValue="signin">
            <TabsList className="grid grid-cols-2 w-full mb-6">
              <TabsTrigger value="signin">{c.signIn}</TabsTrigger>
              <TabsTrigger value="signup">{c.signUp}</TabsTrigger>
            </TabsList>

            <TabsContent value="signin">
              <form onSubmit={handleSignIn} className="space-y-4">
                <div>
                  <Label htmlFor="si-email">{c.email}</Label>
                  <Input id="si-email" type="email" required value={siEmail} onChange={(e) => setSiEmail(e.tar
get.value)} />
                </div>
                <div>
                  <Label htmlFor="si-pass">{c.password}</Label>
                  <Input id="si-pass" type="password" required minLength={6} value={siPass} onChange={(e) => s
etSiPass(e.target.value)} />
                </div>
                <Button type="submit" variant="hero" size="xl" className="w-full" disabled={loading}>
                  {c.submitIn}
                </Button>
              </form>
            </TabsContent>

            <TabsContent value="signup">
              <form onSubmit={handleSignUp} className="space-y-4">
                <div>
                  <Label>{c.role}</Label>
                  <div className="grid grid-cols-2 gap-2 mt-1">
                    <Button type="button" variant={role === "worker" ? "hero" : "outline"} onClick={() => setR
ole("worker")}>
                      {c.worker}
                    </Button>
                    <Button type="button" variant={role === "owner" ? "hero" : "outline"} onClick={() => setRo
le("owner")}>
                      {c.owner}
                    </Button>
                  </div>
                </div>
                <div>
                  <Label htmlFor="name">{c.name}</Label>
                  <Input id="name" required maxLength={100} value={name} onChange={(e) => setName(e.target.val
ue)} />
                </div>
                <div className="grid grid-cols-2 gap-3">
                  <div>
                    <Label htmlFor="phone">{c.phone}</Label>
                    <Input id="phone" required maxLength={20} value={phone} onChange={(e) => setPhone(e.target
.value)} />
                  </div>
                  <div>
                    <Label htmlFor="area">{c.area}</Label>
                    <Input id="area" required maxLength={100} value={area} onChange={(e) => setArea(e.target.v
alue)} />
                  </div>
                </div>
                <div>
                  <Label htmlFor="su-email">{c.email}</Label>
                  <Input id="su-email" type="email" required value={suEmail} onChange={(e) => setSuEmail(e.tar
get.value)} />
                </div>
                <div>
```

```
                  <Label htmlFor="su-pass">{c.password}</Label>
                  <Input id="su-pass" type="password" required minLength={6} value={suPass} onChange={(e) => s
etSuPass(e.target.value)} />
                </div>
                <Button type="submit" variant="hero" size="xl" className="w-full" disabled={loading}>
                  {c.submitUp}
                </Button>
              </form>
            </TabsContent>
          </Tabs>
        </Card>
      </main>
    </div>
  );
};

export default Auth;
```

src/pages/Dashboard.tsx

```
import { useEffect, useState, useCallback } from "react";
import { useNavigate, Link } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Card } from "@/components/ui/card";
import { Switch } from "@/components/ui/switch";
import { Badge } from "@/components/ui/badge";
import { Avatar, AvatarImage, AvatarFallback } from "@/components/ui/avatar";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from "@/components/ui/dialog";
import { toast } from "sonner";
import { Languages, Sparkles, LogOut, Phone, MapPin, Star, Loader2, ThumbsUp, Camera, User as UserIcon } from
"lucide-react";
import { tDash, type Lang } from "@/lib/i18n";
import type { User } from "@supabase/supabase-js";

type Profile = {
  id: string; full_name: string; phone: string; area: string; role: "worker" | "owner";
};
type Worker = {
  id: string; skills: string[]; day_rate: number; available_today: boolean;
  rating: number; rating_count: number; bio: string; photo_url: string | null;
  profiles?: { full_name: string; phone: string; area: string } | null;
};

const Dashboard = () => {
  const nav = useNavigate();
  const [lang, setLang] = useState<Lang>("kn");
  const c = tDash[lang];
  const fontClass = lang === "kn" ? "font-kannada" : "";

  const [loading, setLoading] = useState(true);
  const [profile, setProfile] = useState<Profile | null>(null);

  // worker form
  const [skills, setSkills] = useState("");
  const [rate, setRate] = useState<number>(500);
  const [bio, setBio] = useState("");
  const [available, setAvailable] = useState(false);
  const [photoUrl, setPhotoUrl] = useState<string | null>(null);
  const [uploadingPhoto, setUploadingPhoto] = useState(false);
  const [saving, setSaving] = useState(false);
  const [savedWorker, setSavedWorker] = useState<{ skills: string[]; day_rate: number; bio: string; available_
today: boolean; updated_at: string; photo_url: string | null } | null>(null);

  // browse
  const [workers, setWorkers] = useState<Worker[]>([]);
  const [onlyAvail, setOnlyAvail] = useState(true);
  const [myRatings, setMyRatings] = useState<Set<string>>(new Set());
  const [viewWorker, setViewWorker] = useState<Worker | null>(null);



  const ensureProfile = useCallback(async (user: User): Promise<Profile | null> => {
    const { data: existing } = await supabase.from("profiles").select("*").eq("id", user.id).maybeSingle();
    if (existing) return existing as Profile;

    const meta = user.user_metadata || {};
    const fallbackProfile = {
      id: user.id,
      full_name: String(meta.full_name || user.email?.split("@")[0] || ""),
      phone: String(meta.phone || ""),
      area: String(meta.area || ""),
      role: (meta.role === "worker" ? "worker" : "owner") as "worker" | "owner",
    };
    const { data: created, error } = await supabase.from("profiles").insert(fallbackProfile).select("*").singl
e();
    if (error) {
      toast.error(c.profileMissing);
      return null;
    }
    return created as Profile;
  }, [c.profileMissing]);
  const loadWorker = useCallback(async (uid: string) => {
```

```
    const { data } = await supabase.from("workers").select("*").eq("id", uid).maybeSingle();
    if (data) {
      setSkills(data.skills.join(", "));
      setRate(data.day_rate);
      setAvailable(data.available_today);
      setBio((data as any).bio || "");
      setPhotoUrl((data as any).photo_url || null);
      setSavedWorker({
        skills: data.skills,
        day_rate: data.day_rate,
        bio: (data as any).bio || "",
        available_today: data.available_today,
        updated_at: (data as any).updated_at,
        photo_url: (data as any).photo_url || null,
      });
    } else {
      setSavedWorker(null);
    }
  }, []);

  const loadAll = useCallback(async () => {
    const { data } = await supabase
      .from("workers")
      .select("id, skills, day_rate, available_today, rating, rating_count, bio, photo_url, profiles(full_name
, phone, area)")
      .order("available_today", { ascending: false })
      .order("rating", { ascending: false });
    setWorkers((data as any) || []);
  }, []);

  const loadMyRatings = useCallback(async (uid: string) => {
    const { data } = await supabase.from("ratings").select("worker_id").eq("owner_id", uid);
    setMyRatings(new Set((data || []).map((r: any) => r.worker_id)));
  }, []);

  useEffect(() => {
    let mounted = true;
    const init = async () => {
      const { data: s } = await supabase.auth.getSession();
      if (!s.session) { nav("/auth", { replace: true }); return; }
      const uid = s.session.user.id;
      const p = await ensureProfile(s.session.user);
      if (!mounted) return;
      setProfile(p as Profile);
      if (p?.role === "worker") await loadWorker(uid);
      else { await loadAll(); await loadMyRatings(uid); }
      setLoading(false);
    };
    init();

    const { data: sub } = supabase.auth.onAuthStateChange((_e, session) => {
      if (!session) nav("/auth", { replace: true });
    });
    return () => { mounted = false; sub.subscription.unsubscribe(); };
  }, [nav, ensureProfile, loadWorker, loadAll, loadMyRatings]);

  // Realtime: refresh the browse list when any worker changes
  useEffect(() => {
    if (profile?.role !== "owner") return;
    const ch = supabase
      .channel("workers-rt")
      .on("postgres_changes", { event: "*", schema: "public", table: "workers" }, () => loadAll())
      .subscribe();
    return () => { supabase.removeChannel(ch); };
  }, [profile?.role, loadAll]);

  const handleSave = async () => {
    if (!profile) return;
    setSaving(true);
    const skillArr = skills.split(",").map((s) => s.trim()).filter(Boolean).slice(0, 10);
    const { error } = await supabase.from("workers").upsert({
      id: profile.id,
      skills: skillArr,
      day_rate: Math.max(0, Math.min(99999, Math.floor(rate))),
      available_today: available,
      bio: bio.slice(0, 280),
    });
    setSaving(false);
```

```
  if (error) toast.error(error.message);
  else {
    toast.success(c.saved);
    await loadWorker(profile.id);
  }
};

const handlePhotoUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
  const file = e.target.files?.[0];
  if (!file || !profile) return;
  const allowed: Record<string, string> = {
    "image/jpeg": "jpg",
    "image/png": "png",
    "image/webp": "webp",
    "image/gif": "gif",
  };
  if (!allowed[file.type]) { toast.error("Only JPEG, PNG, WebP or GIF images are allowed"); return; }
  if (file.size > 5 * 1024 * 1024) { toast.error("Max 5MB"); return; }
  setUploadingPhoto(true);
  const ext = allowed[file.type];
  const path = `${profile.id}/avatar.${ext}`;
  const { error: upErr } = await supabase.storage
    .from("worker-photos")
    .upload(path, file, { upsert: true, contentType: file.type });
  if (upErr) { setUploadingPhoto(false); toast.error(upErr.message); return; }
  const { data: pub } = supabase.storage.from("worker-photos").getPublicUrl(path);
  const url = `${pub.publicUrl}?t=${Date.now()}`;
  const skillArr = skills.split(",").map((s) => s.trim()).filter(Boolean).slice(0, 10);
  const { error: dbErr } = await supabase.from("workers").upsert({
    id: profile.id, skills: skillArr, day_rate: Math.floor(rate),
    available_today: available, bio: bio.slice(0, 280), photo_url: url,
  });
  setUploadingPhoto(false);
  if (dbErr) { toast.error(dbErr.message); return; }
  setPhotoUrl(url);
  toast.success(c.photoUpdated);
  await loadWorker(profile.id);
};

const toggleAvailable = async (val: boolean) => {
  if (!profile) return;
  setAvailable(val);
  const skillArr = skills.split(",").map((s) => s.trim()).filter(Boolean).slice(0, 10);
  const { error } = await supabase.from("workers").upsert({
    id: profile.id, skills: skillArr, day_rate: Math.floor(rate),
    available_today: val, bio: bio.slice(0, 280),
  });
  if (error) { toast.error(error.message); setAvailable(!val); }
};

const toggleRating = async (workerId: string) => {
  if (!profile) return;
  const has = myRatings.has(workerId);
  const next = new Set(myRatings);
  if (has) {
    next.delete(workerId);
    setMyRatings(next);
    const { error } = await supabase.from("ratings").delete()
      .eq("worker_id", workerId).eq("owner_id", profile.id);
    if (error) { toast.error(error.message); next.add(workerId); setMyRatings(new Set(next)); }
    else toast.success(c.unrated);
  } else {
    next.add(workerId);
    setMyRatings(next);
    const { error } = await supabase.from("ratings").insert({
      worker_id: workerId, owner_id: profile.id, value: 1,
    });
    if (error) { toast.error(error.message); next.delete(workerId); setMyRatings(new Set(next)); }
    else toast.success(c.rated);
  }
  loadAll();
};

const logout = async () => { await supabase.auth.signOut(); };

if (loading) {
  return <div className="min-h-screen flex items-center justify-center bg-gradient-warm">
    <Loader2 className="w-8 h-8 animate-spin text-primary" />
```

```
    </div>;
  }

  if (!profile) {
    return <div className={`min-h-screen bg-gradient-warm flex items-center justify-center p-6 ${fontClass}`}>
      <Card className="max-w-md p-6 text-center bg-gradient-card border-2 shadow-warm">
        <p className="mb-4 text-muted-foreground">{c.profileMissing}</p>
        <Button variant="hero" onClick={logout}>{c.logout}</Button>
      </Card>
    </div>;
  }

  const visibleWorkers = onlyAvail ? workers.filter((w) => w.available_today) : workers;

  return (
    <div className={`min-h-screen bg-gradient-warm ${fontClass}`}>
      <nav className="container flex items-center justify-between py-6">
        <Link to="/" className="flex items-center gap-2">
          <div className="w-10 h-10 rounded-xl bg-gradient-hero flex items-center justify-center shadow-warm">
            <Sparkles className="w-5 h-5 text-primary-foreground" />
          </div>
          <span className="text-xl font-bold">■■■-■■■■</span>
        </Link>
        <div className="flex items-center gap-2">
          <span className="hidden sm:inline text-sm font-medium">{c.hi}, {profile?.full_name}</span>
          <Button variant="outline" size="sm" onClick={() => setLang(lang === "kn" ? "en" : "kn")}>
            <Languages className="w-4 h-4" />{lang === "kn" ? "English" : "■■■■■"}
          </Button>
          <Button variant="outline" size="sm" onClick={logout}>
            <LogOut className="w-4 h-4" />{c.logout}
          </Button>
        </div>
      </nav>

      <main className="container pb-16">
        {profile?.role === "worker" ? (
          <Card className="max-w-2xl mx-auto p-8 shadow-warm bg-gradient-card border-2 animate-slide-up">
            <h2 className="text-2xl font-extrabold mb-6">{c.workerTab}</h2>

            <div className="mb-6 flex items-center gap-4">
              <Avatar className="w-24 h-24 border-2 border-primary/30">
                {photoUrl ? <AvatarImage src={photoUrl} alt={profile.full_name} /> : null}
                <AvatarFallback className="bg-accent/30">
                  <UserIcon className="w-10 h-10 text-muted-foreground" />
                </AvatarFallback>
              </Avatar>
              <Label htmlFor="photo" className="cursor-pointer">
                <div className="inline-flex items-center gap-2 px-4 py-2 rounded-lg bg-primary text-primary-fo
reground hover:opacity-90 text-sm font-medium">
                  {uploadingPhoto ? <Loader2 className="w-4 h-4 animate-spin" /> : <Camera className="w-4 h-4"
 />}
                  {uploadingPhoto ? c.uploading : (photoUrl ? c.changePhoto : c.uploadPhoto)}
                </div>
                <input id="photo" type="file" accept="image/*" className="hidden"
                  onChange={handlePhotoUpload} disabled={uploadingPhoto} />
              </Label>
            </div>

            <div className="mb-6 rounded-2xl border bg-card p-4">
              <h3 className="font-bold mb-3">{c.accountInfo}</h3>
              <div className="grid sm:grid-cols-2 gap-3 text-sm">
                <div><span className="text-muted-foreground">{c.name}:</span> <span className="font-medium">{p
rofile.full_name || "—"}</span></div>
                <div><span className="text-muted-foreground">{c.phone}:</span> <span className="font-medium">{
profile.phone || "—"}</span></div>
                <div><span className="text-muted-foreground">{c.area}:</span> <span className="font-medium">{p
rofile.area || "—"}</span></div>
                <div><span className="text-muted-foreground">{c.role}:</span> <span className="font-medium">{p
rofile.role === "worker" ? c.worker : c.owner}</span></div>
              </div>
            </div>

            <div className="mb-6 rounded-2xl border-2 border-primary/30 bg-card p-4">
              <h3 className="font-bold mb-3 flex items-center gap-2">
                <Star className="w-4 h-4 text-primary" />{c.savedProfile}
              </h3>
              {savedWorker ? (
                <div className="space-y-2 text-sm">
```

```
                <div><span className="text-muted-foreground">{c.skills}:</span>{" "}
                  {savedWorker.skills.length ? (
                    <span className="font-medium">{savedWorker.skills.join(", ")}</span>
                  ) : <span className="text-muted-foreground">—</span>}
                </div>
                <div><span className="text-muted-foreground">{c.rate}:</span>{" "}
                  <span className="font-medium">₹{savedWorker.day_rate}</span>
                </div>
                <div><span className="text-muted-foreground">{c.bio}:</span>{" "}
                  <span className="font-medium">{savedWorker.bio || "—"}</span>
                </div>
                <div><span className="text-muted-foreground">{c.availToday}:</span>{" "}
                  <span className="font-medium">{savedWorker.available_today ? c.on : c.off}</span>
                </div>
                <div className="text-xs text-muted-foreground pt-1">
                  {c.savedAt}: {new Date(savedWorker.updated_at).toLocaleString()}
                </div>
              </div>
            ) : (
              <p className="text-sm text-muted-foreground">{c.notSavedYet}</p>
            )}
          </div>

          <div className="flex items-center justify-between p-5 rounded-2xl bg-gradient-hero text-primary-fo
reground mb-6 shadow-warm">
            <div>
              <div className="text-sm opacity-90">{c.availToday}</div>
              <div className="text-2xl font-bold">{available ? c.on : c.off}</div>
            </div>
            <Switch checked={available} onCheckedChange={toggleAvailable} className="scale-150" />
          </div>

          <div className="space-y-4">
            <div>
              <Label htmlFor="skills">{c.skills}</Label>
              <Input id="skills" value={skills} onChange={(e) => setSkills(e.target.value)}
                placeholder={lang === "kn" ? "ಪೇಂಟಿಂಗ್, ಗಾರ್ಡನಿಂಗ್" : "Cleaning, Gardening"} maxLength={200}
/>
            </div>
            <div>
              <Label htmlFor="rate">{c.rate}</Label>
              <Input id="rate" type="number" min={0} max={99999} value={rate}
                onChange={(e) => setRate(Number(e.target.value) || 0)} />
            </div>
            <div>
              <Label htmlFor="bio">{c.bio}</Label>
              <Textarea id="bio" value={bio} onChange={(e) => setBio(e.target.value)}
                placeholder={c.bioPh} maxLength={280} rows={3} />
            </div>
            <Button variant="hero" size="xl" className="w-full" onClick={handleSave} disabled={saving}>
              {saving ? <Loader2 className="w-4 h-4 animate-spin" /> : c.save}
            </Button>
          </div>

          <div className="mt-6 p-4 rounded-xl bg-card border text-sm text-muted-foreground flex gap-3">
            <MapPin className="w-4 h-4 shrink-0 mt-0.5 text-primary" />
            <div>{profile.area} · {profile.phone}</div>
          </div>
        </Card>
      ) : (
        <div className="max-w-4xl mx-auto">
          <Card className="mb-6 p-5 bg-gradient-card border-2 shadow-soft">
            <h2 className="text-xl font-extrabold mb-3">{c.accountInfo}</h2>
            <div className="grid sm:grid-cols-4 gap-3 text-sm">
              <div><span className="text-muted-foreground">{c.name}:</span> <span className="font-medium">{p
rofile.full_name || "—"}</span></div>
              <div><span className="text-muted-foreground">{c.phone}:</span> <span className="font-medium">{
profile.phone || "—"}</span></div>
              <div><span className="text-muted-foreground">{c.area}:</span> <span className="font-medium">{p
rofile.area || "—"}</span></div>
              <div><span className="text-muted-foreground">{c.role}:</span> <span className="font-medium">{c
.owner}</span></div>
            </div>
          </Card>

          <div className="flex items-center justify-between mb-6">
            <h2 className="text-2xl font-extrabold">{c.browseTab}</h2>
            <label className="flex items-center gap-2 text-sm font-medium cursor-pointer">
```

```
              <Switch checked={onlyAvail} onCheckedChange={setOnlyAvail} />
              {c.onlyAvail}
            </label>
          </div>

          {visibleWorkers.length === 0 ? (
            <Card className="p-12 text-center text-muted-foreground bg-gradient-card">
              {c.noWorkers}
            </Card>
          ) : (
            <div className="grid sm:grid-cols-2 gap-4">
              {visibleWorkers.map((w) => {
                const rated = myRatings.has(w.id);
                return (
                  <Card key={w.id} className="p-6 bg-gradient-card border-2 hover:border-primary/40 hover:-t
ranslate-y-1 transition-all shadow-soft hover:shadow-warm">
                    <div className="flex items-start gap-3 mb-3">
                      <Avatar className="w-14 h-14 border-2 border-primary/20 shrink-0">
                        {w.photo_url ? <AvatarImage src={w.photo_url} alt={w.profiles?.full_name || ""} /> :
 null}
                        <AvatarFallback className="bg-accent/30">
                          <UserIcon className="w-6 h-6 text-muted-foreground" />
                        </AvatarFallback>
                      </Avatar>
                      <div className="flex-1 min-w-0">
                        <h3 className="font-bold text-lg truncate">{w.profiles?.full_name || "—"}</h3>
                        <div className="text-xs text-muted-foreground flex items-center gap-1 mt-1">
                          <MapPin className="w-3 h-3" />{w.profiles?.area}
                        </div>
                      </div>
                      <Badge variant={w.available_today ? "default" : "secondary"} className={w.available_to
day ? "bg-secondary" : ""}>
                        {w.available_today ? c.available : c.notAvailable}
                      </Badge>
                    </div>

                    {w.bio && <p className="text-sm text-muted-foreground mb-3 line-clamp-2">{w.bio}</p>}

                    <div className="flex flex-wrap gap-1 mb-3">
                      {w.skills.slice(0, 5).map((s, i) => (
                        <span key={i} className="text-xs px-2 py-1 rounded-full bg-accent/30 text-foreground
">{s}</span>
                      ))}
                    </div>

                    <div className="flex items-center justify-between mb-4">
                      <div className="text-sm flex items-center gap-1">
                        <Star className="w-4 h-4 fill-accent text-accent" />
                        <span className="font-bold">{Number(w.rating).toFixed(1)}</span>
                        <span className="text-xs text-muted-foreground">· {w.rating_count} {c.reviews}</span
>
                      </div>
                      <div className="text-lg font-extrabold text-primary">
                        ■{w.day_rate}<span className="text-xs text-muted-foreground font-normal">/{lang ===
"kn" ? "■■■" : "day"}</span>
                      </div>
                    </div>

                    <div className="grid grid-cols-3 gap-2 mb-2">
                      <Button variant="hero" size="lg" className="col-span-2" asChild disabled={!w.profiles?
.phone}>
                        <a href={`tel:${w.profiles?.phone}`}>
                          <Phone className="w-4 h-4" />{c.call}
                        </a>
                      </Button>
                      <Button
                        variant={rated ? "default" : "outline"}
                        size="lg"
                        onClick={() => toggleRating(w.id)}
                        title={rated ? c.thumbed : c.thumbUp}
                        aria-label={c.thumbUp}
                      >
                        <ThumbsUp className={`w-4 h-4 ${rated ? "fill-current" : ""}`} />
                      </Button>
                    </div>
                    <Button variant="outline" size="sm" className="w-full" onClick={() => setViewWorker(w)}>
                      <UserIcon className="w-4 h-4" />{c.viewProfile}
                    </Button>
```

```
                  </Card>
                );
              })}
            </div>
          )}
        </div>
      )}
    </main>

    <Dialog open={!!viewWorker} onOpenChange={(o) => !o && setViewWorker(null)}>
      <DialogContent className={`max-w-md ${fontClass}`}>
        <DialogHeader>
          <DialogTitle>{c.workerProfile}</DialogTitle>
        </DialogHeader>
        {viewWorker && (
          <div className="space-y-4">
            <div className="flex items-center gap-4">
              <Avatar className="w-20 h-20 border-2 border-primary/30">
                {viewWorker.photo_url ? <AvatarImage src={viewWorker.photo_url} alt={viewWorker.profiles?.fu
ll_name || ""} /> : null}
                <AvatarFallback className="bg-accent/30">
                  <UserIcon className="w-8 h-8 text-muted-foreground" />
                </AvatarFallback>
              </Avatar>
              <div>
                <h3 className="text-xl font-bold">{viewWorker.profiles?.full_name || "—"}</h3>
                <div className="text-sm text-muted-foreground flex items-center gap-1 mt-1">
                  <MapPin className="w-3 h-3" />{viewWorker.profiles?.area || "—"}
                </div>
                <div className="text-sm flex items-center gap-1 mt-1">
                  <Star className="w-4 h-4 fill-accent text-accent" />
                  <span className="font-bold">{Number(viewWorker.rating).toFixed(1)}</span>
                  <span className="text-xs text-muted-foreground">· {viewWorker.rating_count} {c.reviews}</s
pan>
                </div>
              </div>
            </div>

            <div className="text-sm space-y-2">
              <div>
                <span className="text-muted-foreground">{c.skills}:</span>{" "}
                <span className="font-medium">{viewWorker.skills.join(", ") || "—"}</span>
              </div>
              <div>
                <span className="text-muted-foreground">{c.rate}:</span>{" "}
                <span className="font-extrabold text-primary">■{viewWorker.day_rate}</span>
              </div>
              <div>
                <span className="text-muted-foreground">{c.availToday}:</span>{" "}
                <span className="font-medium">{viewWorker.available_today ? c.available : c.notAvailable}</s
pan>
              </div>
              {viewWorker.bio && (
                <div>
                  <div className="text-muted-foreground">{c.bio}:</div>
                  <p className="font-medium mt-1">{viewWorker.bio}</p>
                </div>
              )}
              {viewWorker.profiles?.phone && (
                <div>
                  <span className="text-muted-foreground">{c.phone}:</span>{" "}
                  <span className="font-medium">{viewWorker.profiles.phone}</span>
                </div>
              )}
            </div>
          </div>
        )}
        <DialogFooter className="gap-2 sm:gap-2">
          {viewWorker?.profiles?.phone && (
            <Button variant="hero" asChild>
              <a href={`tel:${viewWorker.profiles.phone}`}>
                <Phone className="w-4 h-4" />{c.call}
              </a>
            </Button>
          )}
          <Button variant="outline" onClick={() => setViewWorker(null)}>{c.close}</Button>
        </DialogFooter>
      </DialogContent>
```

```
      </Dialog>
    </div>
  );
};

export default Dashboard;
```

src/pages/NotFound.tsx

```
import { useLocation } from "react-router-dom";
import { useEffect } from "react";

const NotFound = () => {
  const location = useLocation();

  useEffect(() => {
    console.error("404 Error: User attempted to access non-existent route:", location.pathname);
  }, [location.pathname]);

  return (
    <div className="flex min-h-screen items-center justify-center bg-muted">
      <div className="text-center">
        <h1 className="mb-4 text-4xl font-bold">■■■</h1>
        <p className="mb-4 text-xl text-muted-foreground">■■■■■! ■■■ ■■■■■■■■</p>
        <a href="/" className="text-primary underline hover:text-primary/90">
          ■■■■■■■■■■ ■■■■■■■■■
        </a>
      </div>
    </div>
  );
};

export default NotFound;
```

src/lib/i18n.ts

```
export type Lang = "kn" | "en";

export const tAuth = {
  kn: {
    title: "███-█████████ ███████",
    sub: "███████ █████ ████ ███ ████ █████████",
    signIn: "███████", signUp: "███████",
    email: "██████", password: "███████████",
    name: "██████ ██████", phone: "█████ ███████", area: "███████ / ████",
    role: "█████…", worker: "████████", owner: "██████",
    submitIn: "███████ ████", submitUp: "████ █████████",
    back: "██████",
  },
  en: {
    title: "Welcome to Mane-Kelsa",
    sub: "Sign in or create a new account",
    signIn: "Sign In", signUp: "Sign Up",
    email: "Email", password: "Password",
    name: "Full Name", phone: "Phone Number", area: "Area / Street",
    role: "I am a…", worker: "Worker", owner: "Owner",
    submitIn: "Sign In", submitUp: "Create Account",
    back: "Back",
  },
} as const;

export const tDash = {
  kn: {
    hi: "████████", logout: "████ ███",
    workerTab: "████ ████████", browseTab: "██████████",
    availToday: "████ ████ ████", on: "███", off: "███",
    skills: "██████████ (███████████ ██████████)",
    rate: "████ ██ (■)", save: "██████", saved: "███████████!",
    available: "████ ████", notAvailable: "██████████",
    call: "███ ████", noWorkers: "███████ ██████████ ██████████",
    onlyAvail: "████ ████ ████ ████████",
    rupees: "■/███",
    bio: "████ ██████ (████ ██████)",
    bioPh: "███: ■ ██████ ██████, ████████ ████████ ██████████.",
    trust: "████████",
    rated: "████████! ████████ ██████████",
    unrated: "████████ ██████████████████",
    thumbUp: "███████████████",
    thumbed: "████ ████ ████████████",
    reviews: "████████████████",
    accountInfo: "███████ ███████",
    name: "██████",
    phone: "█████",
    area: "████████",
    role: "██████",
    worker: "█████████",
    owner: "██████",
    profileMissing: "██████ ████████ ██████████████████. ████████ ██████ ███████ ████.",
    savedProfile: "██████ ███████ ████████",
    notSavedYet: "██████ ███ ████████. ██████ ██████ ██████ '██████' ████████.",
    savedAt: "██████ ████ ███████████",
    photo: "█████",
    uploadPhoto: "████ ████████ ████",
    changePhoto: "████ ████████",
    uploading: "████████ ██████████...",
    photoUpdated: "████ ███████████████",
    viewProfile: "████████ ████",
    workerProfile: "████████ ████████",
    close: "███████",
  },
  en: {
    hi: "Hi", logout: "Log Out",
    workerTab: "My Profile", browseTab: "Workers",
    availToday: "I'm available today", on: "ON", off: "OFF",
    skills: "Skills (comma separated)",
    rate: "Day Rate (■)", save: "Save", saved: "Saved!",
    available: "Available today", notAvailable: "Not available",
    call: "Call", noWorkers: "No workers available right now",
    onlyAvail: "Only show available today",
    rupees: "■/day",
    bio: "About me (short intro)",
    bioPh: "e.g. 5 years experience, always on time.",
```

```
    trust: "Trust",
    rated: "Thanks! Rating submitted",
    unrated: "Rating removed",
    thumbUp: "Good work",
    thumbed: "You rated this worker",
    reviews: "reviews",
    accountInfo: "Saved information",
    name: "Name",
    phone: "Phone",
    area: "Area",
    role: "Role",
    worker: "Worker",
    owner: "Owner",
    profileMissing: "Your profile is being prepared. Please sign in again.",
    savedProfile: "Your saved profile",
    notSavedYet: "Nothing saved yet. Fill the form below and press Save.",
    savedAt: "Last saved",
    photo: "Photo",
    uploadPhoto: "Upload photo",
    changePhoto: "Change photo",
    uploading: "Uploading...",
    photoUpdated: "Photo updated",
    viewProfile: "View profile",
    workerProfile: "Worker profile",
    close: "Close",
  },
} as const;
```

src/lib/utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

src/components/NavLink.tsx

```
import { NavLink as RouterNavLink, NavLinkProps } from "react-router-dom";
import { forwardRef } from "react";
import { cn } from "@/lib/utils";

interface NavLinkCompatProps extends Omit<NavLinkProps, "className"> {
  className?: string;
  activeClassName?: string;
  pendingClassName?: string;
}

const NavLink = forwardRef<HTMLAnchorElement, NavLinkCompatProps>(
  ({ className, activeClassName, pendingClassName, to, ...props }, ref) => {
    return (
      <RouterNavLink
        ref={ref}
        to={to}
        className={({ isActive, isPending }) =>
          cn(className, isActive && activeClassName, isPending && pendingClassName)
        }
        {...props}
      />
    );
  },
);

NavLink.displayName = "NavLink";

export { NavLink };
```

src/hooks/use-mobile.tsx

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined);

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !!isMobile;
}
```

src/hooks/use-toast.ts

```
import * as React from "react";

import type { ToastActionElement, ToastProps } from "@/components/ui/toast";

const TOAST_LIMIT = 1;
const TOAST_REMOVE_DELAY = 1000000;

type ToasterToast = ToastProps & {
  id: string;
  title?: React.ReactNode;
  description?: React.ReactNode;
  action?: ToastActionElement;
};

const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
} as const;

let count = 0;

function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER;
  return count.toString();
}

type ActionType = typeof actionTypes;

type Action =
  | {
      type: ActionType["ADD_TOAST"];
      toast: ToasterToast;
    }
  | {
      type: ActionType["UPDATE_TOAST"];
      toast: Partial<ToasterToast>;
    }
  | {
      type: ActionType["DISMISS_TOAST"];
      toastId?: ToasterToast["id"];
    }
  | {
      type: ActionType["REMOVE_TOAST"];
      toastId?: ToasterToast["id"];
    };

interface State {
  toasts: ToasterToast[];
}

const toastTimeouts = new Map<string, ReturnType<typeof setTimeout>>();

const addToRemoveQueue = (toastId: string) => {
  if (toastTimeouts.has(toastId)) {
    return;
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId);
    dispatch({
      type: "REMOVE_TOAST",
      toastId: toastId,
    });
  }, TOAST_REMOVE_DELAY);

  toastTimeouts.set(toastId, timeout);
};

export const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case "ADD_TOAST":
      return {
        ...state,
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
```

```
      };

    case "UPDATE_TOAST":
      return {
        ...state,
        toasts: state.toasts.map((t) => (t.id === action.toast.id ? { ...t, ...action.toast } : t)),
      };

    case "DISMISS_TOAST": {
      const { toastId } = action;

      // ! Side effects ! - This could be extracted into a dismissToast() action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId);
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id);
        });
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t,
        ),
      };
    }
    case "REMOVE_TOAST":
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        };
      }
      return {
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      };
  }
};

const listeners: Array<(state: State) => void> = [];

let memoryState: State = { toasts: [] };

function dispatch(action: Action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) => {
    listener(memoryState);
  });
}

type Toast = Omit<ToasterToast, "id">;

function toast({ ...props }: Toast) {
  const id = genId();

  const update = (props: ToasterToast) =>
    dispatch({
      type: "UPDATE_TOAST",
      toast: { ...props, id },
    });
  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id });

  dispatch({
    type: "ADD_TOAST",
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss();
```

```
      },
    },
  });

  return {
    id: id,
    dismiss,
    update,
  };
}

function useToast() {
  const [state, setState] = React.useState<State>(memoryState);

  React.useEffect(() => {
    listeners.push(setState);
    return () => {
      const index = listeners.indexOf(setState);
      if (index > -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId?: string) => dispatch({ type: "DISMISS_TOAST", toastId }),
  };
}

export { useToast, toast };
```

src/integrations/supabase/client.ts

```
// This file is automatically generated. Do not edit it directly.
import { createClient } from '@supabase/supabase-js';
import type { Database } from './types';

const SUPABASE_URL = import.meta.env.VITE_SUPABASE_URL;
const SUPABASE_PUBLISHABLE_KEY = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;

// Import the supabase client like this:
// import { supabase } from "@/integrations/supabase/client";

export const supabase = createClient<Database>(SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, {
  auth: {
    storage: localStorage,
    persistSession: true,
    autoRefreshToken: true,
  }
});
```

supabase/migrations/20260504115455_970c4592-b84e-4b74-a0db-ea50c842b695.sql

```
-- Role enum
CREATE TYPE public.user_role AS ENUM ('worker', 'owner');

-- Profiles table
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  full_name TEXT NOT NULL DEFAULT '',
  phone TEXT NOT NULL DEFAULT '',
  area TEXT NOT NULL DEFAULT '',
  role public.user_role NOT NULL DEFAULT 'owner',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Authenticated users can view profiles"
  ON public.profiles FOR SELECT TO authenticated USING (true);

CREATE POLICY "Users can insert own profile"
  ON public.profiles FOR INSERT TO authenticated WITH CHECK (auth.uid() = id);

CREATE POLICY "Users can update own profile"
  ON public.profiles FOR UPDATE TO authenticated USING (auth.uid() = id);

-- Workers table
CREATE TABLE public.workers (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  skills TEXT[] NOT NULL DEFAULT '{}',
  day_rate INTEGER NOT NULL DEFAULT 0,
  photo_url TEXT,
  available_today BOOLEAN NOT NULL DEFAULT false,
  rating NUMERIC(2,1) NOT NULL DEFAULT 5.0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE public.workers ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Authenticated users can view workers"
  ON public.workers FOR SELECT TO authenticated USING (true);

CREATE POLICY "Worker can insert own row"
  ON public.workers FOR INSERT TO authenticated WITH CHECK (auth.uid() = id);

CREATE POLICY "Worker can update own row"
  ON public.workers FOR UPDATE TO authenticated USING (auth.uid() = id);

-- updated_at trigger function
CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;

CREATE TRIGGER profiles_updated_at BEFORE UPDATE ON public.profiles
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();
CREATE TRIGGER workers_updated_at BEFORE UPDATE ON public.workers
  FOR EACH ROW EXECUTE FUNCTION public.set_updated_at();

-- Auto-create profile on signup
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
  INSERT INTO public.profiles (id, full_name, phone, area, role)
  VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'full_name', ''),
    COALESCE(NEW.raw_user_meta_data->>'phone', ''),
    COALESCE(NEW.raw_user_meta_data->>'area', ''),
    COALESCE((NEW.raw_user_meta_data->>'role')::public.user_role, 'owner')
  );
  RETURN NEW;
END; $$;

CREATE TRIGGER on_auth_user_created AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

supabase/migrations/20260504115519_2f96c0fb-1030-4969-82ef-f85743725ba2.sql

```
CREATE OR REPLACE FUNCTION public.set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql SET search_path = public AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;

REVOKE EXECUTE ON FUNCTION public.handle_new_user() FROM PUBLIC, anon, authenticated;
REVOKE EXECUTE ON FUNCTION public.set_updated_at() FROM PUBLIC, anon, authenticated;
```

supabase/migrations/20260504122914_def23943-92ee-4874-879d-aff1070e0892.sql

```
-- Add bio and photo to workers
ALTER TABLE public.workers
  ADD COLUMN IF NOT EXISTS bio TEXT NOT NULL DEFAULT '',
  ADD COLUMN IF NOT EXISTS rating_count INTEGER NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS rating_sum INTEGER NOT NULL DEFAULT 0;

-- Ratings table: owners give a 1 (thumbs up) per worker (toggleable)
CREATE TABLE IF NOT EXISTS public.ratings (
  id UUID NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  worker_id UUID NOT NULL,
  owner_id UUID NOT NULL,
  value INTEGER NOT NULL DEFAULT 1 CHECK (value IN (1)),
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
  UNIQUE (worker_id, owner_id)
);

ALTER TABLE public.ratings ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Anyone authenticated can view ratings"
  ON public.ratings FOR SELECT TO authenticated USING (true);

CREATE POLICY "Owner inserts own rating"
  ON public.ratings FOR INSERT TO authenticated
  WITH CHECK (auth.uid() = owner_id);

CREATE POLICY "Owner deletes own rating"
  ON public.ratings FOR DELETE TO authenticated
  USING (auth.uid() = owner_id);

-- Recompute trust score on rating change
CREATE OR REPLACE FUNCTION public.recalc_worker_rating()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  wid UUID;
  cnt INTEGER;
  sm INTEGER;
BEGIN
  wid := COALESCE(NEW.worker_id, OLD.worker_id);
  SELECT COUNT(*), COALESCE(SUM(value),0) INTO cnt, sm FROM public.ratings WHERE worker_id = wid;
  UPDATE public.workers
    SET rating_count = cnt,
        rating_sum = sm,
        rating = CASE WHEN cnt = 0 THEN 5.0 ELSE LEAST(5.0, 4.0 + (sm::numeric / GREATEST(cnt,1))) END
    WHERE id = wid;
  RETURN COALESCE(NEW, OLD);
END; $$;

DROP TRIGGER IF EXISTS ratings_recalc ON public.ratings;
CREATE TRIGGER ratings_recalc
  AFTER INSERT OR DELETE ON public.ratings
  FOR EACH ROW EXECUTE FUNCTION public.recalc_worker_rating();
```

supabase/migrations/20260504122939_d771950b-e4f2-4df2-ae01-ebd60cb30140.sql

```
REVOKE EXECUTE ON FUNCTION public.recalc_worker_rating() FROM PUBLIC, anon, authenticated;
```

supabase/migrations/20260507001640_f7afb6b7-dd33-4965-aa15-e762ae504811.sql

```
-- Create public bucket for worker profile photos
INSERT INTO storage.buckets (id, name, public)
VALUES ('worker-photos', 'worker-photos', true)
ON CONFLICT (id) DO NOTHING;

-- Public read
CREATE POLICY "Worker photos are publicly accessible"
ON storage.objects FOR SELECT
USING (bucket_id = 'worker-photos');

-- Worker can upload own photo (path starts with their uid)
CREATE POLICY "Workers can upload own photo"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (
  bucket_id = 'worker-photos'
  AND auth.uid()::text = (storage.foldername(name))[1]
);

-- Worker can update own photo
CREATE POLICY "Workers can update own photo"
ON storage.objects FOR UPDATE
TO authenticated
USING (
  bucket_id = 'worker-photos'
  AND auth.uid()::text = (storage.foldername(name))[1]
);

-- Worker can delete own photo
CREATE POLICY "Workers can delete own photo"
ON storage.objects FOR DELETE
TO authenticated
USING (
  bucket_id = 'worker-photos'
  AND auth.uid()::text = (storage.foldername(name))[1]
);
```

supabase/migrations/20260507001708_e49da20b-9188-496c-87ee-d473d5276472.sql

```
DROP POLICY IF EXISTS "Worker photos are publicly accessible" ON storage.objects;
```

supabase/migrations/20260515172646_07adf3bd-0835-4db0-89fe-682d48534cdc.sql

```
-- 1. Security definer to avoid recursion in profiles policy
CREATE OR REPLACE FUNCTION public.current_user_role()
RETURNS public.user_role
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public
AS $$ SELECT role FROM public.profiles WHERE id = auth.uid() $$;

-- 2. Profiles SELECT: own profile, or owners can read worker profiles
DROP POLICY IF EXISTS "Authenticated users can view profiles" ON public.profiles;

CREATE POLICY "Users view own profile"
ON public.profiles FOR SELECT TO authenticated
USING (auth.uid() = id);

CREATE POLICY "Owners view worker profiles"
ON public.profiles FOR SELECT TO authenticated
USING (
  public.current_user_role() = 'owner'
  AND EXISTS (SELECT 1 FROM public.workers w WHERE w.id = profiles.id)
);

-- 3. Workers INSERT requires the user to actually be a worker
DROP POLICY IF EXISTS "Worker can insert own row" ON public.workers;
CREATE POLICY "Worker can insert own row"
ON public.workers FOR INSERT TO authenticated
WITH CHECK (
  auth.uid() = id
  AND public.current_user_role() = 'worker'
);

-- 4. Prevent direct rating tampering via a BEFORE UPDATE trigger.
--    Allow rating changes only when recalc_worker_rating sets the flag.
CREATE OR REPLACE FUNCTION public.protect_worker_ratings()
RETURNS trigger LANGUAGE plpgsql SET search_path = public AS $$
BEGIN
  IF current_setting('app.allow_rating_update', true) IS DISTINCT FROM 'on' THEN
    NEW.rating := OLD.rating;
    NEW.rating_count := OLD.rating_count;
    NEW.rating_sum := OLD.rating_sum;
  END IF;
  RETURN NEW;
END $$;

DROP TRIGGER IF EXISTS trg_protect_worker_ratings ON public.workers;
CREATE TRIGGER trg_protect_worker_ratings
BEFORE UPDATE ON public.workers
FOR EACH ROW EXECUTE FUNCTION public.protect_worker_ratings();

-- 5. Update recalc to set the flag so legitimate rating updates pass through
CREATE OR REPLACE FUNCTION public.recalc_worker_rating()
RETURNS trigger LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  wid UUID; cnt INTEGER; sm INTEGER;
BEGIN
  wid := COALESCE(NEW.worker_id, OLD.worker_id);
  SELECT COUNT(*), COALESCE(SUM(value),0) INTO cnt, sm
    FROM public.ratings WHERE worker_id = wid;
  PERFORM set_config('app.allow_rating_update', 'on', true);
  UPDATE public.workers
    SET rating_count = cnt,
        rating_sum = sm,
        rating = CASE WHEN cnt = 0 THEN 5.0
                      ELSE LEAST(5.0, 4.0 + (sm::numeric / GREATEST(cnt,1))) END
    WHERE id = wid;
  PERFORM set_config('app.allow_rating_update', 'off', true);
  RETURN COALESCE(NEW, OLD);
END $$;

-- Ensure recalc trigger exists on ratings
DROP TRIGGER IF EXISTS trg_recalc_worker_rating ON public.ratings;
CREATE TRIGGER trg_recalc_worker_rating
AFTER INSERT OR UPDATE OR DELETE ON public.ratings
FOR EACH ROW EXECUTE FUNCTION public.recalc_worker_rating();

-- 6. Storage: restrict worker-photos uploads to image MIME types
DROP POLICY IF EXISTS "Workers can upload own photo" ON storage.objects;
CREATE POLICY "Workers can upload own photo"
```

```
ON storage.objects FOR INSERT TO authenticated
WITH CHECK (
  bucket_id = 'worker-photos'
  AND (auth.uid())::text = (storage.foldername(name))[1]
  AND lower(coalesce(metadata->>'mimetype','')) IN ('image/jpeg','image/png','image/webp','image/gif')
);

DROP POLICY IF EXISTS "Workers can update own photo" ON storage.objects;
CREATE POLICY "Workers can update own photo"
ON storage.objects FOR UPDATE TO authenticated
USING (
  bucket_id = 'worker-photos'
  AND (auth.uid())::text = (storage.foldername(name))[1]
)
WITH CHECK (
  bucket_id = 'worker-photos'
  AND (auth.uid())::text = (storage.foldername(name))[1]
  AND lower(coalesce(metadata->>'mimetype','')) IN ('image/jpeg','image/png','image/webp','image/gif')
);
```

supabase/migrations/20260515172705_5d4f314d-3eef-401e-82fe-2087e16dcd20.sql

```
REVOKE EXECUTE ON FUNCTION public.current_user_role() FROM PUBLIC, anon;
GRANT EXECUTE ON FUNCTION public.current_user_role() TO authenticated;
```